

Frontend Scaling Risks

Overview

The Android client is functional and feature-rich, but several implementation choices will become increasingly expensive as feed, livestream, chat, and monetization traffic continue to grow.

Major risks

1. oversized feature owners

- ``LiveStreamActivity``, ``ChatActivity``, and ``YenkasaPlayerView`` still own too much behavior each
- regression risk rises as new realtime and media features land

2. singleton-heavy runtime

- ``SocketManager``, ``ApiClient``, ``TokenManager``, and app-wide notification helpers simplify access
- they also make ownership, lifecycle, and mutation paths less explicit

3. feed complexity concentration

- feed caching, socket updates, monetization, player lifecycle, community scoping, and chrome state all meet in one surface
- this increases stale-state and update-order risk

4. push and routing sprawl

- push suppression, sound handling, deep links, and notification navigation are implemented across several layers
- payload shape drift can break user routing silently

5. source-coded secrets and environment coupling

- Cloudinary, maps, and push identifiers are still visible in client code or manifest metadata
- this raises operational and security debt

Known hot paths

- immersive feed scroll and playback
- chat realtime updates and media rendering
- livestream join/reconnect flow
- app bootstrap and notification initialization

Recommended near-term improvements

1. reduce production logging noise and sensitive payload exposure
2. keep breaking large screens into domain coordinators
3. isolate media session management from view code
4. enforce stricter client payload contracts for notifications and realtime events
5. continue completing localization and hardcoded-string removal